

Ch12_Web Application Security

OBJECT:

1. XSS
2. CSRF
3. SQL injection
4. Transport Layer Security

Overview of Django's security features and advice on securing a Django powered site.

1. Cross Site Scripting (XSS) 跨站脚本攻击

Cross Site Scripting (XSS)

- **Cross Site Scripting (XSS)** is found in Web applications that fail to escape user-submitted content properly before rendering it into HTML.
- This allows an attacker to insert arbitrary HTML into your Web page, usually in the form of `<script>` tags --- that is, inject client side scripts into the browsers of other users.
- Attackers often use XSS attacks to steal cookie and session information, or to trick users into giving private information to the wrong person (aka *phishing*).
- Let's take a look at a typical example. We can easily insert `<script>` tags that will run some client-side script!

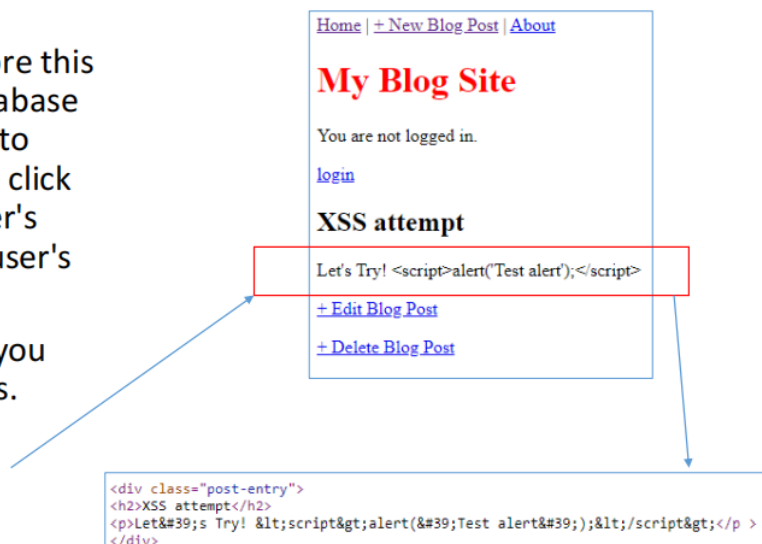


跨站脚本攻击 是一种出现在 Web 应用中的安全漏洞，通常由于应用程序未能正确地转义用户提交的内容，导致恶意的 HTML 或脚本代码（如 `<script>` 标签）被注入到页面中并在其他用户的浏览器中执行。

- 在用户的浏览器中执行任意代码，破坏用户体验甚至导致数据泄露。
- 攻击者可以通过 XSS 窃取用户的 cookie 和会话信息，进而伪装成合法用户。
- 通过诱导用户访问伪装的页面进行 钓鱼（Phishing），从而获取用户的私密信息。
- 如果恶意脚本被存储到数据库中，并在其他用户访问页面时重新显示，就会触发跨站脚本攻击。攻击者的脚本可能会被执行，导致用户的浏览器受到威胁。

Django's protection against XSS

- The problem gets worse if you store this data (malicious scripts) in the database and later retrieved and displayed to other users, or by getting users to click a link which will cause the attacker's JavaScript to be executed by the user's browser.
- Using Django templates protects you against the majority of XSS attacks.
- Django templates escape specific characters which are particularly dangerous to HTML.



< Django templates protects you against the majority of XSS attacks. >

- Django 模板会自动对特定的危险字符进行转义

例如，将 `<` 转义为 `<`，将 `>` 转义为 `>`；=> 即使用户提交了危险的标签，页面只会显示转移后的字符，不会真正执行脚本。

2. Cross Site Request Forgery (CSRF) 跨站请求伪造攻击

Cross Site Request Forgery (CSRF)

- **Cross Site Request Forgery (CSRF)** attacks allow a malicious user to execute actions using the credentials of another user without that user's knowledge or consent.
- For example consider the case where we have a hacker who wants to create additional blog posts for our application.
- A more ambitious hacker could use the same approach on other sites to perform much more harmful tasks (e.g. transfer money to their own accounts, etc.)
- Django has built-in protection against most types of CSRF attacks, providing you have enabled and used it where appropriate.
- CSRF protection works by checking for a nonce in each POST request.
- This ensures that a malicious user cannot simply replay a form POST to your website and have another logged in user unwittingly submit that form. The malicious user would have to know the nonce.

跨站请求伪造（CSRF） 是一种利用用户身份认证进行恶意操作的攻击类型

攻击者

攻击者会创建一个恶意链接或伪造的网站，这个网站包含特定的代码或请求。

当用户点击这个链接或访问伪造网站时，该代码会自动触发并向合法网站发送请求。

这些请求通常是伪装的，不会引起用户的注意，但会利用用户已经登录合法网站的认证信息（如 cookie）

用户

用户通常处于登录状态，并且浏览器中保存了该合法网站的凭证（例如身份验证 cookie 或会话令牌）。

攻击者设计的恶意请求会在用户不知情的情况下自动发送，借助这些凭证让合法网站认为请求来自用户本人。

攻击者可以利用 CSRF 技术代替用户在网站上执行操作，例如：

- 创建额外的博客帖子。
- 更严重的情况是，攻击者伪造请求转移资金到自己的账户。

< Django has built-in protection against it >

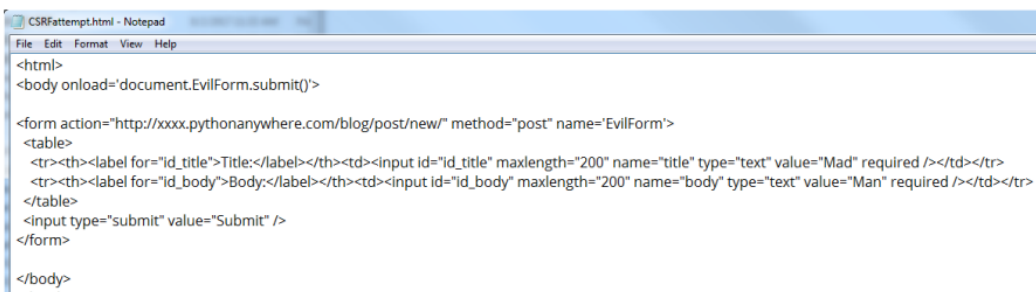
- Django 在每个 POST 请求中检查一个 **nonce**（一次性随机值）。
- 这个随机值嵌入到表单中，确保每次请求都带有唯一的验证标识。
- 恶意用户无法简单地重放表单 POST 请求来冒充合法用户，因为他们无法获得正确的 nonce 值。

CSRF example

```
from django.db import models

# Create your models here
class Post(models.Model):    # add the Post model
    title = models.CharField(max_length=200)
    author = models.ForeignKey('auth.User', on_delete=models.CASCADE,)
    body = models.TextField()
    def __str__(self):
        return self.title
```

- In order to do this, they create an HTML file, which contains an post-creation form that is submitted as soon as the file is loaded. They would then send out the file.
- If the file is opened by any logged in user, then the form would be submitted with their credentials and a new blog post would be created.



```
<html>
<body onload=document.EvilForm.submit()>

<form action="http://xxx.pythonanywhere.com/blog/post/new/" method="post" name="EvilForm">
  <table>
    <tr><th><label for="id_title">Title:</label></th><td><input id="id_title" maxlength="200" name="title" type="text" value="Mad" required /></td></tr>
    <tr><th><label for="id_body">Body:</label></th><td><input id="id_body" maxlength="200" name="body" type="text" value="Man" required /></td></tr>
  </table>
  <input type="submit" value="Submit" />
</form>

</body>
</html>
```

- 攻击者创建了一个 HTML 文件，该文件包含一个博客文章创建表单。

- 文件中的表单会在加载时自动提交 (`<body onload='document.EvilForm.submit()>`)。
- 如果用户已经登录到目标网站并打开了这个文件，攻击者的表单会利用用户的凭证提交请求，创建新的博客文章。

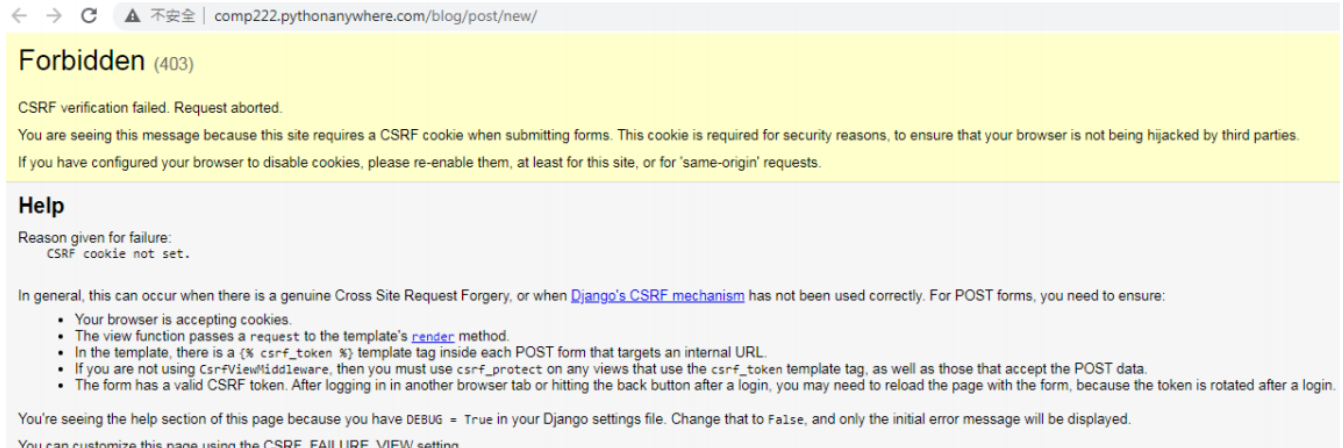
Django's protection against CSRF

- When you open the HTML on the previous slide, you should get a CSRF error, because Django has protection against it.

Title:

Body:

Submit



当你尝试提交 HTML 表单（如之前图片中的恶意表单）时，Django 默认会检测请求的合法性。

如果请求未携带有效的 CSRF Token，Django 会返回 403 Forbidden（请求被拒绝）错误

- Django 在 POST 表单提交时要求附带唯一的 CSRF Token，以确保请求的来源是可信的。
- 如果没有这个 Token，或者 Token 无效，则可能是 CSRF 攻击或表单配置错误，Django 会自动拒绝请求。

Django's protection against CSRF

- The way the protection is enabled is that you include the `{% csrf_token %}` template tag in your form definition.
- This token is then rendered in your HTML as shown below, with a value that is specific to the user on the current browser.

```
<input type='hidden' name='csrfmiddlewaretoken' value='0QRWHnYVg776y2l66mcvZqp8alrv4lb8S8lZ4ZJUWGZFA5VHrVfL2mpH29YZ39PW' />
```
- Django generates a user/browser specific key and will reject forms that do not contain the field, or that contain an incorrect field value for the user/browser.
- To use this type of attack, the hacker now has to discover and include the CSRF key for the specific target user.
- We can prevent man-in-the-middle attack with `CSRF_COOKIE_SECURE` set to True.

Django 通过模板标签 `{% csrf_token %}` 在表单中嵌入一个唯一的安全令牌。

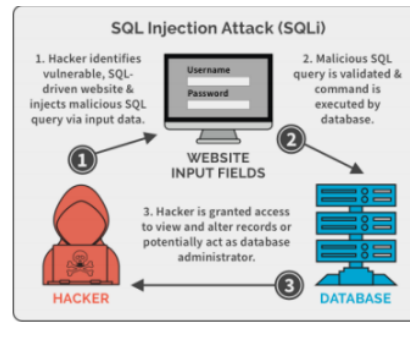
此令牌与用户和浏览器绑定，具有唯一性，确保每次提交的表单都来自合法来源。

通过将 `CSRF_COOKIE_SECURE` 设置为 `True`，令 CSRF Cookie 仅在 HTTPS 连接中传输，进一步增强安全性。

3. SQL injection

SQL injection

- SQL injection is a type of attack where a malicious user is able to execute arbitrary SQL code on a database. This can result in records being deleted or data leakage.



leakage – 泄露

SQL injection

- This vulnerability commonly crops up when constructing SQL “by hand” from user input.
- For example, imagine writing a function to gather a list of contact information from a contact search page.

```
def user_contacts(request):  
    user = request.GET['username']  
    sql = "SELECT * FROM user_contacts WHERE username = '%s';" % username  
    # execute the SQL here...
```

- What happens if an attacker types `" OR 'a'='a'"` into the query box. In that case, the query that the string interpolation will construct will be:

```
SELECT * FROM user_contacts WHERE username = ' OR 'a' = 'a';
```

- Because we allowed unsecured SQL into the string, the attacker's added OR clause ensures that every single row is returned.

SQL 注入 是一种常见的安全漏洞，发生在通过用户输入动态构建 SQL 查询时

攻击者通过注入恶意的 SQL 代码，可能获取未授权的数据或操控数据库。

示例：

```

1 def user_contacts(request):
2     user = request.GET['username']
3     sql = "SELECT * FROM user_contacts WHERE username = '%s';" % username
4     # execute the SQL here...

```

工作原理：从用户输入中接收用户名，并将其直接拼接到 SQL 查询中。

漏洞：如果用户输入恶意数据，SQL 查询的结构会被破坏，可能导致数据泄露。

- 攻击者输入：

```

1 ' OR 'a'='a

```

- 生成的 SQL 语句为：

```

1 SELECT * FROM user_contacts WHERE username = '' OR 'a' = 'a';

```

- **问题：**因为条件 `'a' = 'a'` 总为真，查询结果会返回所有行，而不仅是目标用户的数据。
- Imagine what will happen if the attacker submits `""; DELETE FROM user_contacts WHERE 'a' = 'a'".`

```

SELECT * FROM user_contacts WHERE username = ''; DELETE FROM
user_contacts WHERE 'a' = 'a';

```

Django's protection against SQL injection

- Although this problem is sometimes hard to spot, the solution is simple: *never* trust user-submitted data, and *always* escape it when passing it into SQL.
- By using Django's querysets, the resulting SQL will be properly escaped by the underlying database driver.
- However, Django also gives developers power to write raw queries or execute custom SQL. These capabilities should be used sparingly and you should always be careful to properly escape any parameters that the user can control.

！永远不要信任用户提交的数据。当这些数据需要传递到 SQL 查询时，应确保对它们进行正确的转义 (escaping)，以避免攻击者通过注入恶意 SQL 代码获取或篡改数据。

使用 Django 的 **QuerySets** 来构建查询：

- Django 的 QuerySets 会自动对用户输入的数据进行正确的转义，确保生成的 SQL 语句是安全的。

Django 支持开发者编写原始 SQL 查询或执行自定义 SQL。

- 这些功能应谨慎使用，尤其是在涉及用户输入时，开发者必须确保对所有用户控制的参数进行严格的转义和验证，以避免潜在的安全风险。
-

4. Transport Layer Security (TLS)

Transport Layer Security (TLS)

- TLS is a cryptographic protocol designed to provide communications security over a computer network.
- Its use in securing HTTPS remains the most publicly visible.
- TLS protocol aims primarily to provide security, including privacy (confidentiality), integrity, and authenticity through the use of cryptography, such as the use of certificates, between two or more communicating computer applications.
- TLS builds on the now-deprecated **SSL (Secure Sockets Layer)** specifications. It uses encryption algorithms to scramble data in transit, preventing hackers from reading it as it is sent over the connection.
- In SSL (Secure Socket Layer), the Message digest is used to create a master secret and It provides the basic security services which are **Authentication** and **confidentiality**.

TLS 是什么？

- TLS 是一种加密协议，旨在为计算机网络上的通信提供安全保障。
- 它被广泛用于 HTTPS，保护网页浏览过程中数据的私密性和安全性。

TLS 的安全目标：

- **隐私 (Confidentiality)**：通过加密算法对数据进行加密，确保数据在传输过程中不可被窃听或解读。
- **完整性 (Integrity)**：防止数据在传输过程中被篡改。
- **真实性 (Authenticity)**：通过数字证书验证通信双方的身份，确保与合法服务器通信

与 SSL 的关系：

- TLS 是 SSL（安全套接层协议）的继承者，SSL 已被废弃，现代安全通信中通常使用 TLS。

Transport Layer Security (TLS)

- In TLS (Transport Layer Security), a Pseudo-random function is used to create a master secret.
- TLS works by binding the identities of websites to **cryptographic key pairs** via digital documents known as **X.509 certificates**.
- Each key pair consists of a **private key** and a **public key**. The private key is kept secure, and the public key can be widely distributed via a certificate.
- The special mathematical relationship between the private and public keys in a pair mean that it is possible to use the public key to encrypt a message that can only be decrypted with the private key.
- Furthermore, the holder of the private key can use it to **sign** other digital documents (such as web pages), and anyone with the public key can verify this signature.

18

TLS 使用 X.509 数字证书 将网站的身份与 加密密钥对 绑定。

- 密钥对 包含：
 - 私钥：由所有者保密。
 - 公钥：广泛分发，通常通过证书传递。
- 公钥和私钥具有数学上的特殊关系：
 - 使用公钥加密的消息只能通过私钥解密。
 - 私钥也可以用来对数字文件（如网页）进行签名，公钥验证签名的真实性。

HTTPS

- **HTTPS** (Hyper Text Transfer Protocol Secure) appears in the URL when a website is **secured by TLS**.
- The details of the certificate, including the issuing authority and the corporate name of the website owner, can be viewed by clicking on the lock symbol on the browser bar.
- Users visiting an HTTPS website can be assured of:
 - **Authenticity.** The server presenting the certificate is in possession of the private key that matches the public key in the certificate.
 - **Integrity.** Documents *signed* by the certificate (e.g. web pages) have not been altered in transit by a **man in the middle**.
 - **Encryption.** Communications between the client and server are encrypted.

HTTPS 是 HTTP（超文本传输协议）的安全版本，使用 TLS（传输层安全协议）来加密网络通信。

当一个网站通过 HTTPS 保护时，你会在浏览器的地址栏中看到 `https://` 开头，旁边还有一个锁状图标。

HTTPS 的工作机制

- **数字证书：**
 - HTTPS 网站会使用 **数字证书（例如 X.509 证书）** 来绑定网站身份。
 - 证书中包含网站的所有者信息（例如公司名称）及颁发机构的信息。
 - 浏览器用户可以点击地址栏的锁状图标，查看这些证书细节。
- **私钥和公钥：**
 - 网站拥有与证书绑定的 **私钥** 和 **公钥**。
 - 公钥可以被公开，用于加密消息，而私钥则仅由网站保管，用于解密或签名，确保数据安全。

EXPOSED ERROR MESSAGES

- During development, being able to see tracebacks and errors live in your browser is extremely useful. Django has “pretty” and informative debug messages specifically to make debugging easier.
- However, if these errors get displayed once the site goes live, they can reveal aspects of your code or configuration that could aid an attacker. Django’s philosophy is that site visitors should never see application-related error messages.
- Naturally, of course, developers need to see tracebacks to debug problems in their code. So the framework should hide all error messages from the public, but it should display them to the trusted site developers.
- **The Solution:** uses the simple flag that controls the display of these error messages, i.e. the `DEBUG` setting.

↑ 暴露错误消息（Exposed Error Messages）的重要性 以及 Django 如何在开发和生产环境中处理这些问题。

1. 开发阶段的错误处理

实时调试：在开发环境中，Django 会将错误详情（包括回溯信息、错误类型等）直接显示在浏览器中。

- **好处：**开发者可以立即看到错误的原因，有助于快速调试。
- **风险：**这些详细的错误信息如果在生产环境中暴露，可能会泄露代码或配置细节，给攻击者带来可乘之机。

2. 生产阶段的错误处理

生产环境中，绝不能让网站访问者看到与应用相关的错误消息。

- **风险：**暴露的错误信息可能包含敏感的系统配置（如数据库路径、API 密钥），为攻击者提供了入侵的线索。
- **解决方案：**在生产环境下，Django 默认隐藏这些错误消息，并仅记录错误日志供开发者查看。

3. 使用 DEBUG 设置

- `DEBUG = True`（开发环境）：错误详情会显示在浏览器中，供开发者调试。
- `DEBUG = False`（生产环境）：
 - 错误消息不会直接暴露。
 - 应该配置一个自定义的错误页面（如 404 或 500 页面），以提升用户体验，同时避免信息泄露。

Admin Hardening

- So far it may seem as though the advice is to rely on Django defaults, use HTTPS, add `csrf_token` tags on forms, and set a permissions structure.
- All true. But one step Django does not take on our behalf is hardening the Django admin.
- Consider that every Django website sets the admin, by default, to the `/admin` URL. This is a prime suspect for any hacker trying to access a Django site.
- Therefore an easy step is to simply change the admin URL to literally anything else! E.g. `path('anything-but-admin/', admin.site.urls)`,
- A fun 3rd party package `django-admin-honeypot` will generate a fake admin log in screen and email site admins the IP address of anyone trying to attack your site at `/admin`. These IP addresses can then be added to a blocked address list for the site.
- It's also possible via `django-two-factor-auth` to add two-factor authentication to your admin for an even further layer of protection.

Django 管理面板的安全强化 (Admin Hardening)

默认情况下, Django 的管理面板地址是 `/admin`, 这是攻击者通常首先尝试访问的入口。

建议开发者更改此默认路径, 例如

```
1 path('custom-admin/', admin.site.urls)
```

这样可以降低被恶意扫描工具发现的风险。

使用第三方工具增强安全性

- `django-admin-honeypot`:
 - 该工具可以生成一个假的 admin 登录页面, 吸引攻击者。
 - 当有人尝试访问伪造的登录页面时, 站点管理员会收到攻击者的 IP 地址, 并可以将其加入黑名单。
- `django-two-factor-auth`:
 - 通过启用双因素认证 (2FA), 为 admin 登录增加额外的安全层, 例如需要输入动态验证码。